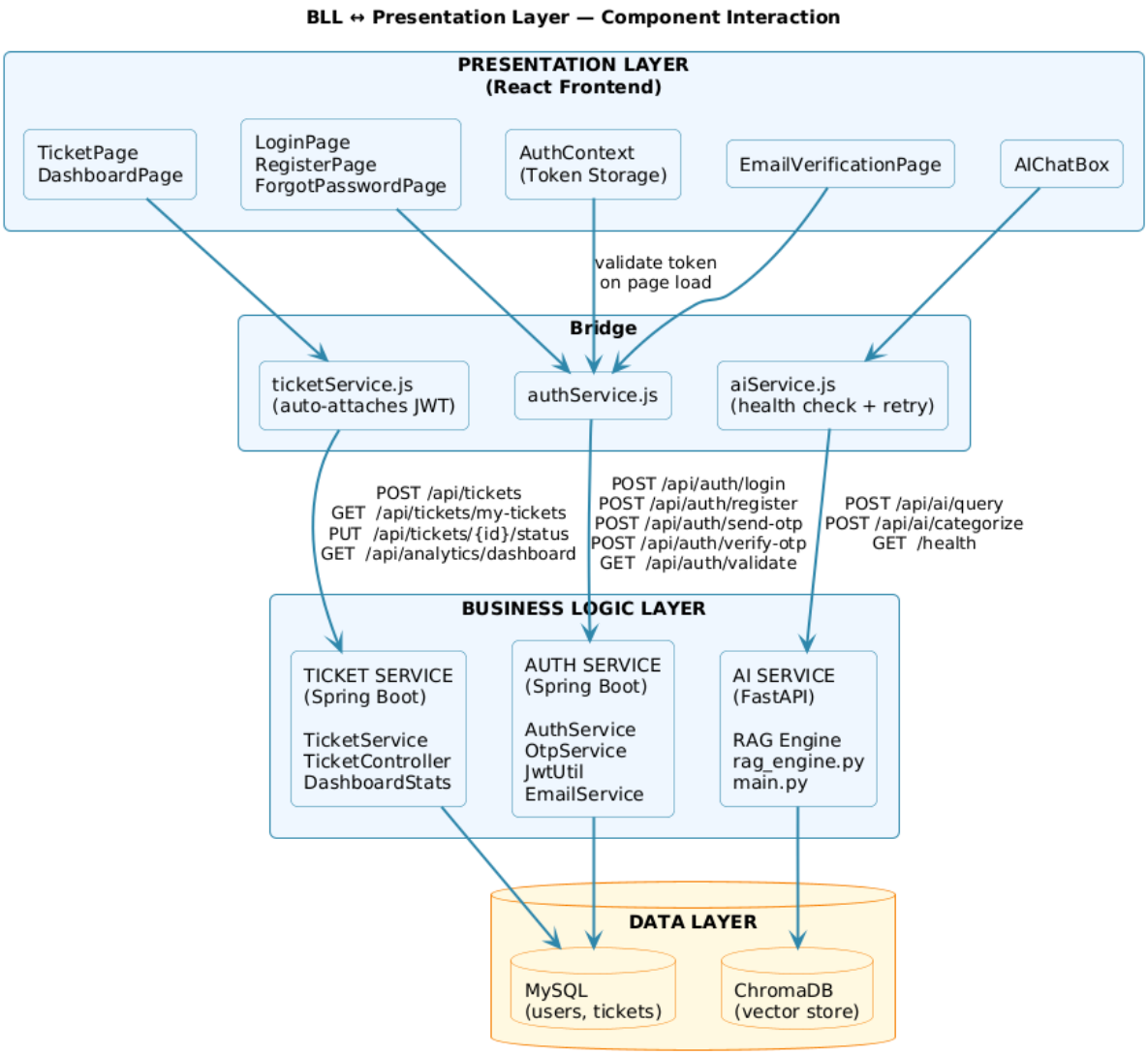
Q1. Core Functional Modules and Their Interaction with the Presentation Layer

The Business Logic Layer (BLL) sits between the user interface and the database. It is responsible for all decisions, what data is allowed, who can access what, and how requests are processed.

In this project, the BLL is split across two backend services: the **Spring Boot Auth Service** (handles users, tickets, and JWT) and the **FastAPI AI Service** (handles intelligent responses and ticket categorization).

The **Presentation Layer** is the React frontend. Users never interact directly with the backend — they interact with the React UI, which communicates with the BLL through three service adapter files: authService.js, ticketService.js, and aiService.js. These files act as the bridge between what the user sees and what the backend processes.

BLL Module	**Presentation Layer Component**
Authentication (AuthService)	LoginPage, RegisterPage
OTP Verification (OtpService)	EmailVerificationPage
Ticket Management (TicketService)	TicketPage, DashboardPage
AI Query / RAG Engine	AIChatBox
Password Reset (AuthService)	ForgotPasswordPage, ResetPasswordPage
JWT Token Management (JwtUtil)	AuthContext (validates on every page load)



Describe the following for your software engineering project.

A) How Are Business Rules Implemented?

Business rules are the conditions the system enforces before allowing any operation to proceed. In this project, they are implemented across six modules.

Authentication Module — Before a new user is saved, the system checks whether the email already exists. If it does, the registration is rejected. At login, the system checks whether the email has been verified — if not, login is blocked regardless of whether the password is correct. Passwords are never stored as plain text; they are stored as a hashed value. During login, BCrypt compares the entered password against the stored hash, and if they do not match, a generic error is shown without revealing which field was wrong.

OTP Management Module — An OTP is exactly 6 digits, generated using a secure random generator. It expires after 5 minutes. Once a user successfully verifies their OTP, it is immediately deleted so it cannot be reused. If a user requests a new OTP before verifying, the old one is overwritten. If an email is already verified, the system refuses to send another OTP.

JWT Token Module — Every token expires after 24 hours. The system automatically validates the token's signature and expiry on every protected request. The secret key used to sign tokens must be at least 256 bits — if it is not, the system refuses to start.

Ticket Management Module — A user can only create a ticket if their email is verified. Users can only view and manage their own tickets. Before any ticket operation is processed, the system extracts the user identity from the JWT token in the request header and checks ownership — if someone tries to access another user's ticket, the request is denied.

Admin Module — All admin endpoints require a special API key sent in the request header. If the key is missing or wrong, the request is immediately rejected with a 403 Forbidden response. When listing users, passwords are always excluded from the response.

AI and RAG Engine Module — When a customer asks a question, only document chunks below a similarity distance threshold are used for the answer — irrelevant chunks are filtered out to keep answers accurate. A confidence score is calculated for every response based on how closely matching the retrieved documents are and the quality of the answer itself. If the confidence falls below 75%, the system marks the response for escalation to a human agent. Ticket category and priority values must match a strict whitelist — if the AI returns anything outside the allowed values, the system falls back to safe defaults.

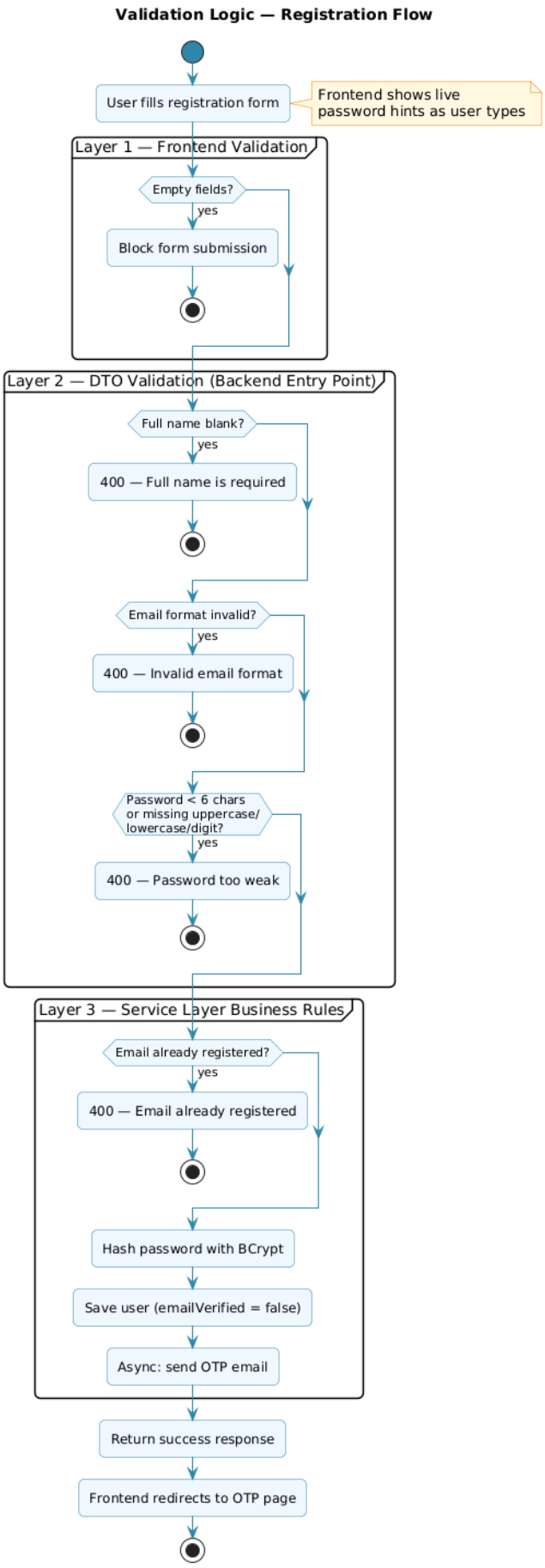
B) Validation Logic

Yes, validation logic is implemented at three separate layers so that incorrect data is caught as early as possible.

Backend DTO Validation — This is the first layer and it runs the moment data arrives from the frontend, before any business logic is executed. For user registration, the full name must not be blank, the email must follow a valid format, and the password must be at least 6 characters and contain at least one uppercase letter, one lowercase letter, and one digit or special character. These rules are enforced using Jakarta Bean Validation annotations directly on the data request objects. For ticket creation, the category, priority, subject, and description fields are all validated for presence and maximum length. The FastAPI AI Service uses Pydantic models which automatically validate that required fields like the query text are present and of the correct data type.

Controller-Level Error Handling — If DTO validation fails, the controllers catch the validation errors and return a structured error response with HTTP status 400, so the frontend always gets a clear, readable message rather than a raw stack trace.

Frontend Validation — The React registration form shows live password requirement hints to the user as they type, giving immediate feedback before the form is even submitted. Login and registration forms use controlled components that prevent empty form submissions. Error messages returned from the backend are displayed inline next to the relevant form fields.


Validation Logic — Registration Flow
User fills registration form
Frontend shows live password hints as user types
Layer 1 — Frontend Validation
Empty fields?
yes
Block form submission
Layer 2 — DTO Validation (Backend Entry Point)
Full name blank?
yes
400 — Full name is required
Email format invalid?
yes
400 — Invalid email format
Password < 6 chars or missing uppercase/ lowercase/digit?
yes
400 — Password too weak
Layer 3 — Service Layer Business Rules
Email already registered?
yes
400 — Email already registered
Hash password with BCrypt
Save user (emailVerified = false)
Async: send OTP email
Return success response
Frontend redirects to OTP page

C) Data Transformation

Data transformation is the process of converting data from the format it is stored in into a format that is safe, clean, and usable by the frontend. In this project, transformation happens at every layer boundary.

Database Entity → Response DTO — The database stores user and ticket data in JPA entity objects which contain internal database IDs, password hashes, lazy-loaded relationships, and JPA annotations. Before any of this is sent to the frontend, it is converted into a clean DTO (Data Transfer Object). For example, the TicketResponse DTO takes a raw Ticket entity and flattens it — instead of sending the full nested User object, it extracts just the customer's email and name as flat fields. This keeps the response simple and hides internal database structure from the client.

Auth Response Transformation — When a user logs in, the AuthResponse DTO is constructed from the user entity. It includes only three fields: the JWT token, the user's full name, and their email. Everything else — the password hash, verification status, internal ID, and creation timestamp — is deliberately excluded so sensitive data never reaches the frontend.

Frontend Storage Transformation — When the frontend receives the login response, it further transforms the data. The JWT token is saved separately in localStorage under its own key. The user details (name and email only) are saved as a JSON object under a separate key. This means the rest of the application can access user details without having to decode the JWT every time.

Dashboard Statistics Transformation — The DashboardStats DTO aggregates multiple separate database queries — counting total tickets, open tickets, in-progress tickets, resolved tickets, and closed tickets — into a single structured object. Instead of making multiple API calls, the frontend receives one clean payload with all the numbers it needs to render the dashboard cards.

AI Confidence to Escalation Flag — The RAG engine produces a raw floating-point confidence score. Before this is sent to the frontend, the AI service transforms it into a simple boolean escalation flag by checking whether the score is below 0.75. This means the frontend does not need to know the threshold or implement any logic — it simply checks one flag and decides whether to show the escalation warning.